

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 9
Operating Systems
Topic: Watching System Calls with strace (Track A - Linux)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: MONITORING COMMAND EXECUTION WITH STRACE **4**

2.1 Tracing the `ls -l` Utility 4

2.2 Answers to Required Observation Questions 4

3 PART 2: DIRECT KERNEL CALLS (`syscalls.c`) **5**

3.1 Implementation of `syscalls.c` 5

3.2 Execution and System Call Trace Verification 5

3.3 Answers to Required Observation Questions 6

4 PART 3: COUNTING SYSTEM CALLS WITH STRACE (-c) **7**

4.1 Aggregated System Call Profiling 7

4.2 Execution and Summary Table Output 7

4.3 Answers to Required Observation Questions 7

4.4 Architectural Analysis: Chunk-Based Ingestion and EOF Detection 8

5 ANALYSIS OF KERNEL-USER BOUNDARIES **9**

5.1 System Call Dispatching Architecture 9

5.2 The Role of `strace` and `ptrace` 9

6 CONCLUSIONS **10**

REFERENCES **11**

1 INTRODUCTION AND GOAL

1.1 Introduction

In modern operating systems adhering to the POSIX specification, user applications execute within unprivileged CPU rings (Ring 3 on x86_64) and are isolated from direct hardware interaction by hardware memory management units (MMUs). Any interaction with peripheral storage, terminal consoles, or IPC pipes requires crossing the hardware privilege boundary into kernel space (Ring 0) via standardized system calls.

While high-level programming interfaces provide abstract wrappers (such as `printf`, `fopen`, and `fread` in the C standard library), these routines ultimately decompose into fundamental kernel system calls (`openat`, `write`, `read`, `close`). The `strace` diagnostic utility leverages the Linux kernel's `ptrace` debugging interface to intercept, decode, and log every system call issued by a process, providing complete transparency into the runtime user-kernel dialogue.

1.2 Lab Objective

The primary objective of this laboratory work is to inspect and analyze the low-level system call interface of Linux using `strace`. Specifically, the work covers:

- tracing the execution of standard GNU utilities (`ls -l`) and identifying exact system calls responsible for directory examination and terminal output;
- developing a standalone C program (`syscalls.c`) that interacts directly with the kernel using core POSIX file descriptor system calls (`open`, `write`, `read`, `close`);
- evaluating the POSIX file descriptor allocation policy and explaining why initial open operations return descriptor `3`;
- analyzing the relationship between user-space buffering libraries (`printf`) and kernel-level output routines (`write`);
- profiling and counting system call distributions using the `strace -c` summary flag, analyzing why file reading requires multiple `read()` invocations, and observing chunk-based buffer reading and End-Of-File (EOF) detection.

2 PART 1: MONITORING COMMAND EXECUTION WITH STRACE

2.1 Tracing the `ls -l` Utility

The standard directory listing utility `ls -l` was executed under `strace`, filtering for system calls associated with descriptor manipulation and data transmission:

```
$ strace -f -e trace=openat,read,write,close ls -l
close(3)                                = 0
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(3)                                = 0
write(1, "total␣28\n-rw-rw-r--␣1␣vitalie␣vi"..., 385) = 385
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Figure 2.1: Figure 2.1 – Selective system call trace of ls -l under strace

2.2 Answers to Required Observation Questions

a) **System Call Used to Display Text:** The `ls` utility employs the **`write`** system call to transmit formatted directory listings to the terminal console:

```
    write(1, "total␣28\ndrwxrwxr-x␣...", 385) = 385
```

Before calling `write`, `ls` queries directory metadata using `openat()` to open the directory file descriptor, `getdents64()` to enumerate directory entries, and `newfstatat()` to retrieve file permissions, timestamps, and sizes.

b) **File Descriptor Written To:** The descriptor number passed to the `write()` system call is strictly **`1`**, representing standard output (`stdout`).

3 PART 2: DIRECT KERNEL CALLS (`syscalls.c`)

3.1 Implementation of `syscalls.c`

A C program was constructed to interact directly with the Linux kernel using POSIX system calls (`open`, `write`, `read`, and `close`) without library-level buffering for file operations:

```c
/* syscalls.c - open, write, read and close, using only system calls.
 * Build: gcc -Wall -Wextra -o syscalls syscalls.c
 * Run:   ./syscalls
 * See the calls: strace ./syscalls
 */
#include <fcntl.h>  /* open */
#include <unistd.h> /* write, read, close */
#include <string.h> /* strlen */
#include <stdio.h>  /* printf */

int main(void) {
    const char *msg = "hello, kernel\n";

    // open() is a system call: ask kernel for file, get back descriptor
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    printf("open returned descriptor %d\n", fd);

    // write() is a system call: transfers bytes into kernel buffer cache
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    // read it back through the kernel
    fd = open("note.txt", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    if (n >= 0) {
        buf[n] = '\0';
    }
    printf("read got %zd bytes: %s", n, buf);
    close(fd);

    return 0;
}
```

Figure 3.1: Figure 3.1 – Source code of syscalls.c

3.2 Execution and System Call Trace Verification

The application was compiled using GCC with strict error checking and executed under `strace`:

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
```

```
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel

$ strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open␣returned␣descriptor␣3\n", 27) = 27
write(3, "hello,␣kernel\n", 14)           = 14
write(1, "write␣sent␣14␣bytes\n", 20)      = 20
close(3)                                  = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello,␣kernel\n", 63)           = 14
write(1, "read␣got␣14␣bytes:␣hello,␣kernel\n", 33) = 33
close(3)                                  = 0
+++ exited with 0 +++
```

Figure 3.2: Figure 3.2 – Intercepted system call trace for ./syscalls

3.3 Answers to Required Observation Questions

a) **File Descriptor Allocation Mechanics (Why Descriptor 3?):**

- Under POSIX operating systems, every process is initialized with three pre-allocated standard streams:
  - `0` — Standard Input (`stdin`);
  - `1` — Standard Output (`stdout`);
  - `2` — Standard Error (`stderr`).
- The Linux kernel follows a strict lowest-available-index allocation policy for process file descriptor tables (`struct files_struct`). When `open()` is invoked, the kernel scans the table starting from index 0 and assigns the first free numerical slot.
- Since slots 0, 1, and 2 are already assigned to terminal descriptors upon process initialization, the first available index is **3**.
- When `close(3)` is called, slot 3 is released. Consequently, the subsequent `open("note.txt", O_RDONLY)` reallocates slot **3**.

b) **Relationship Between `printf()` and `write()`:**

- `printf()` is a user-space library function implemented in the C runtime library (`glibc`). It provides string formatting and user-space memory buffering.
- `write()` is the underlying kernel system call that transfers raw bytes across the user-kernel privilege boundary.
- As seen in Figure 3.2, calling `printf("open returned descriptor 3\n", fd)` does not execute in kernel space directly; instead, `glibc` formats the string into a buffer and immediately invokes `write(1, "open returned descriptor 3\n", 27)` to flush the line to descriptor 1.

4 PART 3: COUNTING SYSTEM CALLS WITH STRACE (-c)

4.1 Aggregated System Call Profiling

In addition to step-by-step stream tracing, `strace` provides a dedicated profiling mode enabled via the `-c` (or `--summary-only`) flag. In this mode, `strace` attaches to the process, intercepts all system calls, measures the execution duration spent within the kernel for each call, and generates an aggregated statistical summary table upon process termination.

To evaluate real file I/O operations, the GNU `cat` utility was profiled while reading the source file `syscalls.c`, redirecting output to `/dev/null` to isolate file reading from console display latency:

```
$ strace -c cat syscalls.c > /dev/null
```

4.2 Execution and Summary Table Output

The command executed and generated the following statistical profile:

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 25.17    0.000037          18         2           read
 19.05    0.000028           7         4           mmap
 14.29    0.000021          10         2           openat
 12.24    0.000018           6         3           close
  9.52    0.000014           7         2           newfstatat
  6.80    0.000010          10         1           write
  4.76    0.000007           7         1           munmap
  4.08    0.000006           6         1           brk
  4.08    0.000006           6         1           execve
  0.00    0.000000           0         1           arch_prctl
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000147                    22           total
```

Figure 4.1: Figure 4.1 – System call count and timing summary generated by strace -c

4.3 Answers to Required Observation Questions

a) **From the summary, how many `read` calls did `cat` make?**

From the summary table, `cat` made exactly **2** `read` system calls.

b) **Explain in one sentence why reading a file needs more than one `read`:**

A file is read in buffer-sized chunks, and the program cannot know when the end of the file is reached until a subsequent `read()` system call returns 0 bytes (the standard POSIX EOF indicator), which requires an extra call after all data has been ingested.

4.4 Architectural Analysis: Chunk-Based Ingestion and EOF Detection

When `cat` reads a file, it allocates a memory buffer according to the file system's block size (typically queried via `fstat`, e.g., 64 KiB or 131,072 bytes).

- **First `read()` call:** The kernel transfers all available bytes of `syscalls.c` (984 bytes) from the kernel page cache into the user buffer. The system call returns 984, representing the number of bytes read.

- **Second `read()` call:** Although the program received fewer bytes than its buffer capacity, the POSIX specification permits short reads even when more data might follow (for instance, on pipes, network sockets, or slow storage). Therefore, the utility must issue another `read()` to definitively check for additional bytes. The kernel identifies that the file pointer has reached the end of the file and returns **0**.

- **EOF Termination:** In Unix/POSIX convention, a return value of `0` from `read()` represents the End-Of-File (EOF) condition. Upon receiving 0, `cat` terminates its read loop and closes the file descriptor.

5 ANALYSIS OF KERNEL-USER BOUNDARIES

5.1 System Call Dispatching Architecture

On the x86_64 Linux architecture, the transition from unprivileged userland to privileged kernel mode proceeds through the `syscall` machine instruction:

a) **Register Setup:** User-space wrappers load the system call number into register `rax` (e.g., `rax = 1` for `SYS_write`, `rax = 257` for `SYS_openat`). Call parameters are loaded sequentially into registers `rdi`, `rsi`, `rdx`, `r10`, `r8`, and `r9`.

b) **Privilege Transition:** The CPU executes the `syscall` instruction. The hardware transitions from Ring 3 to Ring 0, saves the instruction pointer into `rcx`, and jumps to the kernel entry point recorded in the Model Specific Register `IA32_LSTAR`.

c) **Dispatcher Execution:** The kernel entry stub pushes user registers to the kernel stack, checks the system call number against the architecture jump table (`sys_call_table`), and executes the corresponding service routine (such as `ksys_write()`).

d) **Return to Userland:** Upon completing the request, the return value or negative error code is placed into `rax`, and the `sysretq` instruction restores user-space execution.

5.2 The Role of `strace` and `ptrace`

The `strace` utility attaches to the target process using the `ptrace(PTRACE_SYSCALL, ...)` primitive. The kernel suspends the traced process twice per system call: once immediately before syscall entry (allowing inspection of arguments), and once immediately following syscall exit (allowing inspection of return values). This architecture ensures that every transition is captured without altering application source code.

6 CONCLUSIONS

In this laboratory work, the low-level communication interface between user-space applications and the Linux kernel was analyzed using the `strace` diagnostic utility across three comprehensive parts:

a) **Real Command Monitoring (Part 1):** Executing `ls -l` under syscall interception demonstrated that standard utilities depend on basic kernel primitives (`openat`, `getdents64`, `write`) for resource discovery and data rendering.

b) **Direct Kernel Interaction (Part 2):** Developing `syscalls.c` confirmed that file descriptor allocations follow a deterministic lowest-index policy starting at descriptor `3`, and demonstrated that standard formatting functions (`printf`) are user-space abstractions that delegate directly to `write()`.

c) **System Call Profiling (Part 3):** Profiling `cat syscalls.c > /dev/null` with `strace -c` demonstrated quantitative call counting and proved that file ingestion operates in buffer-sized chunks, requiring a terminal 0-byte return to detect the End-Of-File condition.

The experiment highlighted the structural separation of privilege rings, reinforcing the role of the operating system kernel as the sole arbitrator of hardware resources.

REFERENCES

[1] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.

[2] Robert Love, *Linux System Programming: Talking Directly to the Kernel and C Library*, O'Reilly Media, 2nd Edition, 2013.

[3] W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, Addison-Wesley Professional, 3rd Edition, 2013.

[4] Department of Software Engineering and Automation, *Operating Systems Lab 9: Watching System Calls with strace*, FCIM, Technical University of Moldova, 2026.